43

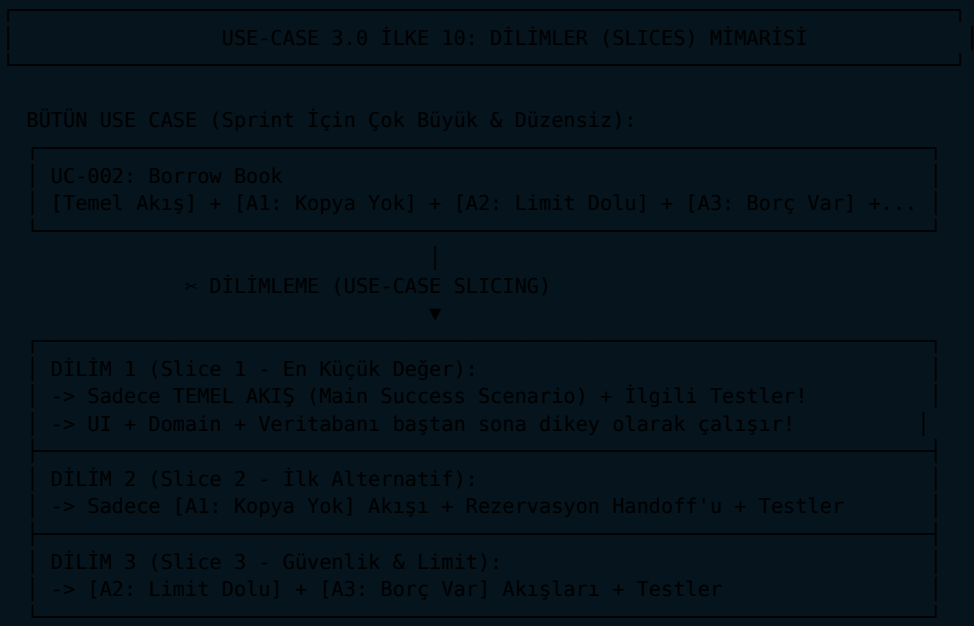
BÖLÜM: DILIMLER (SLICES), TESTLER VE İTERASYON DÖNGÜSÜ | MODULE: SLICES, TESTS, AND THE ITERATION LOOP

10. İlke: Sistemi Dilimlerle İnşa Etmek (Use-Case Slices)

Principle 10: Slices – Slicing the System into Deliverable Units

⚡ Akış, Varlık & Süreç Şeması

⚡ Process & Artifact Schematic



💡 Aptala Anlatım (ELI5 Metaforu)

💡 ELI5 Metaphor

🇹🇷 TÜRKÇE (ELI5)

Kocaman bir doğum günü pastasını bir lokmada ağzınıza atamazsınız, boğulursunuz! Pastayı dilim dilim kesersiniz. İlk dilim sadece kekin ve kremanın tadını verir (Temel akış). Sonraki dilimlerde çilekler ve çikolatalar gelir (Alternatif akışlar). İşte use case pastanın tamamıdır, sprint'e giren şey ise o tek bir lezzetli dilimdir (Slice)!

🇬🇧 ENGLISH (ELI5)

You cannot swallow an entire multi-layered birthday cake in one bite; you'd choke! You slice it cleanly. Slice 1 gives you the foundational sponge cake and frosting (Basic Flow). Subsequent slices bring strawberries and chocolate chips (Alternative Flows). The use case is the whole cake; the actionable backlog ticket for your sprint is the vertical slice!

🔍 MAANG & AIUP Seviyesi Teknik Derinlik

🇹🇷 AIUP & MAANG DERİNLEMESİNE ANALİZ

Use-Case 3.0'ın 10. İlkesi (Build the System in Slices): Ivar Jacobson ve Ian Spence'in klasik use case metodolojisine kattığı en devrimci ilke budur. Bir use case, bir hedefe ulaşmanın TÜM yollarını (başarı, istisnalar, hatalar) içerdiği için tek bir sprint veya tek bir PR için fazla büyüktür ve dengesiz boyutlardadır. Teslimat birimi use case değil, 'Use-Case Slice'dır. Bir dilim; use case içindeki bir veya birkaç yolu, o yolları icra eden tasarımı, kodu ve testleri içeren dikey, bağımsız ve test edilebilir bir pakettir. Bu sayede use case'ler Scrum sprint backlog'larına, Kanban sütunlarına ve SAFe trenlerine mükemmel uyum sağlar. AI Unified Process'te bir şartnamenin Status alanı bu dilimleme yaşam döngüsünü yönetir: Temel akış kodlanıp testten geçtiğinde use case 'Implemented/Tested' aşamasına geçer; kalan alternatif akışlar şartnamede beklerken sistem güvenle çalışır durumda kalır.

🔍 MAANG & AIUP Technical Deep-Dive

🇬🇧 AIUP & MAANG TECHNICAL DEEP-DIVE

Principle 10: Building the System in Slices: Jacobson and Spence added the 10th principle to bridge classical use cases with agile delivery. Because a use case captures every permutation of achieving a goal, it is irregularly sized and too monolithic to flow smoothly through a sprint board. The fundamental unit of delivery is the use-case slice: one or more paths through a use case, accompanied by the design, production code, and automated tests that implement and verify them. Slices start at the trigger, conclude at the postconditions, and provide immediate, testable business value. Slices map directly to user stories or sprint backlog items. In the AI Unified Process, this additive structure allows teams to generate Slice 1 via `/implement UC-002`, review the running UI with stakeholders, and then progressively generate slices for alternative flows without architectural churn.

⚡ Modern Java 25 & Clean Architecture Uygulaması

⚡ Java 25 & Clean Architecture Implementation

```java
// Java 25: Birinci Dilim (Slice 1: Sadece Temel Akış) Uygulaması
package com.maang.aiup.slices;

import java.time.LocalDate;

public class UseCaseSliceOneDemo {
    // DİLİM 1: Sadece temel akış verileri ve başarısı (Alternatifler henüz yok!)
    public record BorrowSliceOneResult(String bookId, String memberId, LocalDate dueDate) {}

    public static class BorrowBookSliceOneService {
        public BorrowSliceOneResult executeBasicFlow(String bookId, String memberId) {
            // Dilim 1 kapsamında sadece mutlu yol çalışır:
            LocalDate dueDate = LocalDate.now().plusDays(28);
            System.out.println("📦 Slice 1 Başarılı: Kitap (" + bookId + ") üyeye (" + memberId + ")
verildi.");
            return new BorrowSliceOneResult(bookId, memberId, dueDate);
        }
    }

    void main() {
        var sliceOne = new BorrowBookSliceOneService();
        var result = sliceOne.executeBasicFlow("BK-42", "MEM-007");
        System.out.println("✅ Dilim 1 Dikey Olarak Çalıştı: " + result);
    }
}
```

⚠️ AI Yanılsaması, Mekanizma Sızıntısı & Mülakat Tuzağı

🇹🇷 MÜLAKAT TUZAĞI VE AI HALÜSİNASYONU

Bir use case'in tüm akışlarını (happy path + 8 alternatif akış) tek bir seferde AI'a kodlatmaya çalışmayın! Temel akışı ilk dilim olarak üretip paydaşa gösterin, geri bildirim alın, sonra diğer akışları ekleyin.

⚠️ AI Hallucination, Mechanism Leak & Interview Trap

🇬🇧 INTERVIEW TRAP & AI HALLUCINATION

Resist the urge to generate an entire use case (happy path + 8 branches) in a single massive prompt. Implement Slice 1 first, get visual feedback, then add alternative branches incrementally.

🎯 Mülakatta ve Üretimde Hatırlanacak Altın Kurallar

- Teslimatın temel birimi use case değil, dikey ve test edilebilir 'use-case slice'dır.
- Dilim 1 daima temel akıştır (happy path); en hızlı iş değerini ve geri bildirimi o sağlar.
- Dilimleme sayesinde use case'ler Scrum, Kanban ve CI/CD akışlarına pürüzsüz entegre olur.

🎯 Key Architectural Takeaways

- The unit of delivery is the use-case slice, not the entire use case document.
- Slice 1 is always the basic flow, delivering immediate feedback and functional baseline.
- Slicing enables seamless integration of formal use cases into Scrum, Kanban, and modern CI/CD.